J
1

Session 1 · Jour 1 / 9

Fondations Kafka & Architecture

FORMATION APACHE KAFKA · 9 JOURS

Apache Kafka de A à Z

Fondations Kafka & Architecture distribuée

Cluster Kafka

3 brokers Docker KRaft

Topics & Partitions

Offset, ISR, réplication RF=3

CLI Kafka complète

kafka-topics · produce · consume

KRaft sans ZooKeeper

Architecture metadata Raft

⏱️ 1 jour · 40% théorie / 60% TP · 50 slides | Prérequis : Docker Desktop · Java 17 · Bases Linux

Plan du Jour 1 — Programme complet (50 slides)

01 — 9h00–10h30
Event-Driven Architecture & Historique — Slides 3–12

02 — 10h45–12h00
Anatomie d'un cluster Kafka — Slides 13–22

03 — 13h00–14h30
Réplication, ISR & Haute Disponibilité — Slides 23–32

04 — 14h30–15h30
KRaft — Architecture sans ZooKeeper — Slides 33–40

05 — 15h45–17h30
TP — Déploiement, CLI & Exercices — Slides 41–50

Pause café 10h30–10h45 · Déjeuner 12h00–13h00 · Pause 15h30–15h45 · Fin 17h30

01

Event-Driven Architecture & Historique Kafka

Du messaging classique au streaming distribué · Pourquoi Kafka s'impose

Histoire d'Apache Kafka — Origines & évolution (2010–2025)

2010

Naissance chez LinkedIn

Jay Kreps, Neha Narkhede et Jun Rao créent Kafka chez LinkedIn pour traiter 1 milliard d'événements/jour. Objectif : remplacer un système de messaging défaillant qui ne tenait pas la charge.

2011

Open-source Apache

LinkedIn donne Kafka à l'Apache Software Foundation. Rapidement adopté par la communauté data engineering pour sa simplicité et ses performances. Version 0.7 publiée.

2014

Confluent est fondé

Jay Kreps, Neha Narkhede et Jun Rao créent Confluent Inc. Ils enrichissent l'écosystème : Schema Registry, Kafka Connect, Kafka Streams, ksqlDB.

2017

Kafka Streams & Connect GA

Kafka Streams production-ready. Kafka Connect avec 100+ connecteurs. Kafka devient la colonne vertébrale des architectures microservices.

2021

KRaft — Adieu ZooKeeper

KIP-500 : KRaft remplace ZooKeeper. Kafka gère ses propres métadonnées en interne via le protocole Raft. Kafka 2.8 : KRaft en preview.

2023+

KRaft GA + Cloud Native

Kafka 3.3+ : KRaft GA. Tiered Storage (S3/GCS). Intégration Flink. 80%+ des Fortune 100 l'utilisent. Standard incontesté du streaming mondial.

LinkedIn utilise toujours Kafka : 7 trillions de messages/jour · Kafka alimente les recommandations de 900M membres.

Les problèmes que Kafka résout — Chaos de l'intégration

Avant Kafka — Intégration point-à-point = chaos N×M connexions

6 systèmes × 5 destinations = 30 connexions !

- App Web
- App Mobile
- ERP
- Analytics
- Data Lake
- CRM

→ **Couplage fort · Cascade de pannes**
→ **Pas de replay · Schemas incompatibles**

Avec Kafka — N+M connexions seulement

Apache Kafka

Producers → Kafka · Kafka → Consumers

✔ Découplage total · Replay · Schema Registry

Les 6 avantages fondamentaux de l'architecture Kafka

✔ Découplage — Producers ignorent les consumers — couplage zéro

✔ Durabilité — Messages persistés sur disque — aucune perte

✔ Scalabilité — Partitions indépendantes — scale horizontal natif

✔ Replay — Rembobiner l'historique depuis n'importe quel offset

✔ Haute dispo — Réplication RF=3 — tolère 1 panne broker

✔ Temps réel — < 5ms end-to-end · millions msg/sec

Event-Driven vs Request/Response — Comparatif détaillé

Critère	Request/Response	Event-Driven Kafka
Couplage	Fort — A connaît B et C	**Faible — A ignore les consommateurs**
Disponibilité	Si B down → A échoue	**Kafka bufferise — B reprend plus tard**
Scalabilité	Chaque hop ajoute de la latence	**Consumers scalent indépendamment**
Débit	Limité par le maillon le plus lent	**Producteur non bloqué**
Replay	Impossible — réponse éphémère	**Oui — seek offset 0, rejoue depuis t=0**
Audit	Difficile — logs éparpillés	**Natif — le topic Kafka EST l'audit log**
Tests	Mocks de toutes les dépendances	**Isolation complète par service**
Use case	API REST, CRUD, login	Streaming, analytics, notifications

Pub/Sub vs Queue — Deux modèles fondamentaux

QUEUE — RabbitMQ, SQS, ActiveMQ

Producer → [Queue] → 1 Consumer

Consommation unique

Un message est lu par 1 SEUL consumer. Supprimé dès acquitté.

Pas de replay

Message acquitté = perdu définitivement. Impossible de relire.

Competing consumers

N consumers se partagent les messages — chacun traite une partie.

Idéal pour les tâches

Job queue, envoi d'emails, traitement images — 1 exécution/message.

PUB/SUB LOG — Apache Kafka

Producer → [Topic Log] → N Consumer Groups

Multi-consumers

N Consumer Groups lisent le MÊME topic — chacun a ses propres offsets.

Replay infini

Seek n'importe quel offset. Historique selon rétention (7j, 1To...).

Ordre par partition

Même clé → même partition → ordre préservé pour chaque entité.

Idéal pour les flux

Streaming, CDC, analytics, sync multi-systèmes, audit log.

Kafka combine les deux : Consumer Groups = comportement queue · Plusieurs groupes = pub/sub · Le meilleur des deux mondes.

Le modèle Log Distribué — Cœur d'Apache Kafka

TOPIC : commerce.orders — Partition 0 — RF=3 — Lecture séquentielle append-only

→ **Immuable · Persisté disque · Replay depuis offset 0 · Ordre garanti dans une partition**

Clé de partitionnement : même clé → même partition → ordre garanti pour chaque entité (commande, client, capteur…)

Kafka vs Solutions concurrentes — Tableau de bord

Critère	Apache Kafka	RabbitMQ	Pulsar	Kinesis
Modèle	**Log distribué partitionné**	Queue AMQP	Log+Queue natif	Log cloud managé
Throughput	**>7M msg/sec**	~50k/sec	~1M/sec	Élevé (quota)
Latence P99	**< 5ms**	< 1ms	< 5ms	~200ms
Rétention	**Illimitée**	Jusqu'à ack	Configurable	7 jours max
Replay	✔ **Natif (seek)**	✗ Non	✔ Natif	✔ 7j seulement
Multi-consumer	✔ **N groupes indép.**	⚠️ Competing	✔ Natif	✔ Partiel
Open Source	✔ **Apache 2.0**	✔ Mozilla	✔ Apache 2.0	✗ Propriétaire
Schema Registry	✔ **Confluent SR**	✗ Non natif	✔ Natif	✗ Non natif

Cas d'usage Apache Kafka — Quand l'utiliser ?

📊 Streaming Analytics temps réel

Calcul de KPIs en continu (CA, NPS, conversion). Fenêtres temporelles Kafka Streams. Résultat visible < 1 seconde. Ex : dashboards temps réel, détection d'anomalies.

📡 IoT & Télémétrie

Ingestion de millions de capteurs (GPS, température). 50k devices × 1msg/sec = 50k/sec. Kafka absorbe sans broncher.

🔄 Intégration SI & CDC

Change Data Capture avec Debezium. Réplication BDD vers d'autres systèmes. PostgreSQL → Elasticsearch, MySQL → S3 Data Lake en temps réel.

🏦 Finance & Trading

Détection de fraude < 50ms. Calcul de risque. Streaming prix de marché. Exactly-once pour transactions critiques.

🔔 Notifications & Events

Événements commande, paiement, livraison. Chaque microservice publie son état. Consumers (email, push, SMS) réagissent en temps réel.

📝 Audit Log & Event Sourcing

Le topic Kafka EST le journal d'audit immuable. Replay complet pour reconstruire l'état à n'importe quelle date.

Règle : si vous avez des flux continus à traiter, distribuer ou archiver → Kafka est presque toujours la bonne réponse.

Kafka en chiffres — État de l'art 2025

Dimension	QUEUE (RabbitMQ, SQS...)	PUB/SUB LOG — Kafka
Consommation	1 msg → 1 consumer (supprimé après)	**1 msg → N Consumer Groups indépendants**
Replay	Impossible — perdu après ack	**Infini — seek offset à tout moment**
Persistance	Jusqu'à acquittement	**Configurable : 7j / 1To / illimité**
Ordre	Par file uniquement	**Garanti par partition**
Scalabilité	Competing consumers	**Partitions indépendantes**
Use case	Job queues · Emails · Tâches async	Streaming · CDC · Analytics · Audit

Quiz — Section 01 : Event-Driven & Historique

❓ Qui a créé Apache Kafka ?

- ○ Amazon Web Services
- ✔ **Jay Kreps, Neha Narkhede & Jun Rao chez LinkedIn**
- ○ La fondation Apache
- ○ Confluent Inc.

❓ Différence clé entre une Queue et Kafka ?

- ○ Kafka est plus rapide
- ✔ **Dans une queue le message est supprimé après lecture. Kafka conserve et permet le replay**
- ○ Kafka ne supporte pas plusieurs consumers
- ○ Une queue est toujours persistée

❓ Quel est l'avantage clé du modèle log distribué ?

- ○ Messages chiffrés automatiquement
- ✔ **Replay : relire l'historique depuis n'importe quel offset**
- ○ Latence toujours < 1ms
- ○ Kafka supporte uniquement JSON

❓ Depuis quelle version Kafka fonctionne sans ZooKeeper ?

- ○ Kafka 2.0
- ✔ **Kafka 2.8 (preview) · 3.3 GA**
- ○ Kafka 1.0
- ○ Kafka 4.0 seulement

Réponses correctes surlignées en vert — Discutez chaque point avec le formateur si un doute persiste.

02

Anatomie d'un cluster Kafka

Broker · Topic · Partition · Offset · Producer · Consumer · Consumer Group

Le Broker Kafka — Rôle et responsabilités en détail

🖥️ Broker Kafka

Un broker est un serveur Kafka (JVM) stockant des partitions sur disque et les distribuant aux producers/consumers. Chaque broker a un node.id unique. Cluster Kafka = ensemble de brokers coordonnés via KRaft.

Stockage

Stocke des partitions en segment files sur disque local (.log + index). Format optimisé pour les accès séquentiels. Zero-copy vers les consumers.

Réplication

Le leader réplique vers les followers ISR. Panne → un follower ISR élu nouveau leader automatiquement en < 30s.

Leadership

Pour chaque partition, un broker est LEADER (toutes R/W). Les autres avec une copie sont FOLLOWERS. Réélu automatiquement si crash.

Controller (KRaft)

3 brokers forment le quorum Controller. Gèrent métadonnées : liste brokers, leaders, configs topics, ACLs via __cluster_metadata.

Le Topic Kafka — Anatomie et configuration

Étape	Nom	Description
1	Sérialisation	Objet Java → bytes. StringSerializer, JsonSerializer, AvroSerializer.
2	Partitionnement	Clé → murmur2(key)%N déterministe. Clé nulle → sticky partitioning.
3	Batch accumulation	Groupé en RecordAccumulator (batch.size=16KB). linger.ms attend.
4	Compression	snappy/lz4/gzip/zstd. -50 à -70% taille réseau. Overhead CPU minimal.
5	Envoi réseau	Sender thread NIO. max.in.flight=5. Ordre garanti avec idempotence.
6	ACK & Retry	acks=0/1/all. retries=MAX_INT avec idempotence. delivery.timeout.ms=120s.

La Partition Kafka — Unité fondamentale de scalabilité

Paramètre	Défaut	Prod.	Description
num.partitions	1	**6–50**	Parallélisme max — impossible à réduire
replication.factor	1	**3**	Nombre de copies de chaque partition
min.insync.replicas	1	**2**	Min. réplicas synchronisés pour écriture
log.retention.hours	168 (7j)	**168–720**	Durée avant suppression des segments
log.retention.bytes	-1 (∞)	100GB+	Taille max du topic · -1 = illimité
cleanup.policy	delete	delete/compact	delete: supprime · compact: garde 1 msg/clé
compression.type	producer	**snappy**	Compression broker — snappy = meilleur compromis

L'Offset Kafka — Mécanisme de positionnement

📍 Offset

Identifiant entier unique et séquentiel (0, 1, 2...) attribué à chaque message dans une partition. Immuable : le message à l'offset 42 sera toujours le même. Les consumers gèrent leur propre offset de lecture.

Log End Offset (LEO)

Position du prochain message à écrire = dernier offset + 1. Représente la taille totale du log. Augmente à chaque écriture.

Committed Offset

Dernier offset confirmé par commitSync/commitAsync. En cas de crash, le consumer repart exactement de là.

Current Offset

Position actuelle du consumer. Il lira le prochain message à cet offset lors du prochain consumer.poll().

Consumer Lag

LAG = LEO - Committed Offset. Mesure le retard de traitement. Alarme si LAG > seuil (ex: 10 000 messages).

```
# Voir les offsets : kafka-consumer-groups.sh --describe --group mon-groupe  →  TOPIC  PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG
```

Topic interne __consumer_offsets stocke les offsets de tous les groupes. Rétention : offsets.retention.minutes=10080 (7j).

Le Producer Kafka — 6 étapes du cycle de vie

1

Sérialisation

Objet Java → bytes. StringSerializer, JsonSerializer, AvroSerializer. Le Deserializer consumer doit correspondre exactement.

2

Partitionnement

Clé non-nulle → murmur2(key) % N. Clé nulle → sticky partitioning (batch.size). Custom Partitioner possible.

3

Accumulation batch

Groupé en RecordAccumulator (batch.size=16KB). linger.ms attend. batch.size atteint → envoi immédiat. Réduit les appels réseau.

4

Compression

snappy : perf/CPU · lz4 : ultra-rapide · gzip : meilleure compression · zstd : excellent (Kafka 2.1+). -50 à -70% taille.

5

Envoi réseau

Sender thread NIO en parallèle. max.in.flight=5 par défaut. Avec idempotence : ordre garanti même avec retries.

6

ACK & Retry

acks=0/1/all. retries=MAX_INT avec idempotence. delivery.timeout.ms=120s. request.timeout.ms=30s.

Config prod : acks=all · enable.idempotence=true · retries=MAX_INT · max.in.flight=5 · compression.type=snappy

Le Consumer Kafka — Poll loop et commit d'offset

Le Poll Loop — Pattern fondamental

Consumer Java — Pattern complet production

```
Properties p = new Properties();
p.put("bootstrap.servers","broker1:9092");
p.put("group.id","orders-processor-v1");
p.put("key.deserializer","...StringDeserializer");
p.put("value.deserializer","...StringDeserializer");
p.put("enable.auto.commit","false"); // CRITIQUE en prod
p.put("max.poll.records","500");
p.put("session.timeout.ms","30000");

KafkaConsumer<String,String> c=new KafkaConsumer<>(p);
c.subscribe(Arrays.asList("commerce.orders"));
try {
  while (running) {
    var records=c.poll(Duration.ofMillis(100));
    for (var r : records) process(r); // votre logique
    c.commitSync(); // commit APRÈS traitement
  }
} finally { c.close(); }
```

Paramètres clés consumer

enable.auto.commit	false en prod
max.poll.records	500
session.timeout.ms	30000
heartbeat.interval.ms	10000
max.poll.interval.ms	300000
fetch.min.bytes	1
fetch.max.wait.ms	500ms

Stratégies de commit d'offset

Auto-commit

enable.auto.commit=true. Simple mais risqué : commit AVANT processing → at-most-once (perte possible).

commitSync()

Bloque jusqu'à confirmation. Plus sûr. Commit après batch → at-least-once (duplication possible si crash).

commitAsync()

Non-bloquant + callback. Meilleur débit. Ne pas utiliser seul (risque commits désordonnés).

Production : commitSync() après chaque batch traité · Combiner commitAsync() en boucle + commitSync() à la fermeture.

Consumer Group — Parallélisme et scénarios

Règle : 1 partition = 1 consumer MAX par groupe à l'instant T — les extras sont IDLE

Scénario	Distribution	Résultat
1 consumer, 6 partitions	C1 traite P0+P1+P2+P3+P4+P5	⚠️ Sous-optimal
3 consumers, 6 partitions	C1:P0+P1 · C2:P2+P3 · C3:P4+P5	Équilibré — 3× plus rapide
6 consumers, 6 partitions	**1 partition par consumer**	**✔ Optimal**
7 consumers, 6 partitions	6 actifs · C7 = IDLE (aucune partition)	⚠️ 7ème consumer inutilisé

Étape	Action	Détail
1	Trigger	Consumer rejoint/quitte, session.timeout.ms dépassé
2	Stop the world	TOUS les consumers stoppent et commitSync() leurs offsets
3	Redistribution	Group Coordinator redistribue les partitions entre consumers actifs
4	Reprise	Consumers reprennent depuis le dernier offset commité

Rebalancing — Redistribution automatique des partitions

🔄 **Rebalancing**

Redistribution automatique des partitions entre consumers d'un groupe quand un consumer rejoint/quitte ou quand session.timeout.ms expire. Pendant le rebalancing, TOUS les consumers stoppent (stop-the-world).

1 — Trigger détecté

Consumer rejoint (subscribe), quitte (close/crash), ou max.poll.interval.ms dépassé. Group Coordinator (broker) détecte.

2 — Stop the World

Le GC envoie REBALANCE_IN_PROGRESS. Tous les consumers commitSync() leurs offsets et stoppent le poll.

3 — JoinGroup

Consumers envoient JoinGroup au GC. Le premier reçoit le rôle Group Leader (consumer, pas broker).

4 — Assignment

Group Leader exécute l'algo (RangeAssignor, RoundRobinAssignor, StickyAssignor) et envoie le plan au GC.

5 — Reprise

GC distribue via SyncGroup. Chaque consumer reçoit ses partitions et reprend le polling depuis le dernier offset commité.

Optimisation : CooperativeStickyAssignor (Kafka 2.4+) = rebalancing incrémental, seules les partitions affectées bougent.

Quiz — Section 02 : Anatomie d'un cluster

❓ 6 partitions, 8 consumers dans un groupe — que se passe-t-il ?

- ○ Tous les 8 consumers reçoivent des messages
- ✔ **6 consumers actifs (1 par partition) · 2 sont IDLE**
- ○ Kafka crée 2 partitions auto
- ○ Erreur — impossible

❓ Différence Current Offset vs Committed Offset ?

- ○ Ils sont toujours identiques
- ✔ **Current = position de lecture · Committed = dernier offset sauvegardé (reprise crash)**
- ○ Committed est toujours plus grand
- ○ Current est géré par le broker

❓ Meilleure config producer pour la durabilité ?

- ○ compression.type=snappy
- ✔ **acks=all + enable.idempotence=true + min.insync.replicas=2**
- ○ batch.size=65536
- ○ max.in.flight=10

❓ enable.auto.commit=true + consumer crash ?

- ○ Aucune perte — Kafka garde tout
- ✔ **Messages commités non traités peuvent être perdus (at-most-once)**
- ○ Reprise exacte depuis dernier message
- ○ Kafka réenvoie tout automatiquement

03

Réplication, ISR & Haute Disponibilité

Leader/Follower · ISR · acks · min.insync.replicas · Scénarios de panne

Réplication Kafka — Leader, Follower et ISR

Topic: orders · Partition 0 · RF=3 · ISR=[1,2,3] — Tolérance : perte de 1 broker

Rôle	Broker 1 — LEADER ★	Broker 2 — FOLLOWER ISR	Broker 3 — FOLLOWER ISR
Fonction	Reçoit toutes les écritures Producer Répond aux lectures Consumer	Réplique en temps réel le Leader Éligible si Broker 1 crash	Réplique en temps réel le Leader Éligible si B1 ou B2 crash
Log (exemple)	OFF:0 OFF:1 OFF:2 OFF:3 OFF:4	OFF:0 OFF:1 OFF:2 OFF:3 OFF:4	OFF:0 OFF:1 OFF:2 OFF:3 OFF:4

acks	Nom	Garantie	Cas d'usage
0	Fire & Forget	Aucune — perte possible si crash	Métriques · Logs
1	Leader ack	Leader seulement — perte si crash avant réplication	Events non-critiques
all	All ISR acks	**Zéro perte* avec min.ISR=2 + RF=3**	Finance · Facturation · CDC

Paramètre acks — Guide complet des garanties producer

acks=0 Fire & Forget	acks=1 Leader Acknowledge	acks=all All ISR Acknowledge
✔ Avantages	✔ Avantages	✔ Avantages

Configuration	Tolérance	Comportement	Usage
RF=3 + min.ISR=1	⚠️ 0 garantie	Écriture avec 1 seul broker ⟶ perte possible	JAMAIS en prod
RF=3 + min.ISR=2 ✔	**✔ 1 broker**	NotEnoughReplicas si 2 brokers tombent (protection)	**Standard prod**
RF=3 + min.ISR=3	0 broker	NotEnoughReplicas si 1 broker lent ⟶ trop strict	Cas ultra-critiques

min.insync.replicas — La garantie de durabilité

🔒 **min.insync.replicas**

Nombre minimum de réplicas dans l'ISR pour accepter une écriture acks=all. Si ISR < seuil → NotEnoughReplicasException. Protège contre la perte de données en cas de pannes multiples simultanées.

RF=3 + min.ISR=1 (dangereux)

Le producer peut écrire si 1 seul broker a reçu le message. Si ce broker crashe avant réplication → données perdues. NE JAMAIS utiliser en production.

RF=3 + min.ISR=2 (recommandé ✔)

Au moins 2 brokers confirment. Tolère 1 broker en panne sans perte. Si 2 brokers tombent → NotEnoughReplicas (protection intentionnelle).

RF=3 + min.ISR=3 (trop strict)

Les 3 brokers doivent être synchronisés. Dès qu'un seul a du retard → NotEnoughReplicas. Disponibilité trop faible. Réservé aux cas ultra-critiques.

```
# Config broker: min.insync.replicas=2  · Config producer: acks=all + retries=MAX  · Config topic: replication.factor=3
```

Ces 3 paramètres forment le triangle de durabilité : RF=3 + min.ISR=2 + acks=all = zéro perte de données garantie.

Scénario de panne — Reprise automatique Kafka (RTO < 30s)

Scénario : crash du broker leader · RF=3, ISR=[1,2,3]

1

État normal

Broker 1 = Leader Partition 0 · Broker 2, 3 = Followers synchronisés · ISR=[1,2,3]

2

Broker 1 crashe

process.kill ou panne réseau · Broker 1 ne répond plus · Controller détecte via heartbeat timeout

3

Élection leader

Controller élit Broker 2 comme nouveau leader · Broker 2 = Leader P0 · ISR=[2,3] · < 10 secondes

4

Reprise auto

Producers redirigent vers Broker 2 · Consumers reprennent depuis dernier offset commité · Service continu

5

Retour Broker 1

Redémarre ⟶ se synchronise depuis Broker 2 · Rejoue messages manqués · Reintègre ISR=[1,2,3] (~30s)

Haute Disponibilité — Configuration et Disaster Recovery

🏗️ Architecture HA minimale en production

- 3 brokers dans 3 Availability Zones différentes (AWS/GCP/Azure) · RF=3 pour tous les topics critiques
- min.insync.replicas=2 · unclean.leader.election.enable=false · auto.leader.rebalance.enable=true

Tolérance aux pannes

- RF=3 + min.ISR=2 : tolère 1 broker
- RF=5 + min.ISR=3 : tolère 2 brokers
- Auto-rebalance leaders : enabled
- Preferred leader election : 300s

Config durabilité

- unclean.leader.election=false ⚠️
- log.flush.interval.messages=10000
- replica.lag.time.max.ms=30000
- replica.fetch.max.bytes=1048576

Disaster Recovery

- MirrorMaker 2 : cross-DC active-passive
- Confluent Cluster Linking : active-active
- RPO quasi-nul entre AZ
- RTO < 30s intra · < 5min cross-DC

Rétention des données — Stratégies et configuration

📅 Rétention Kafka

Kafka conserve les messages sur disque selon une politique configurable. La donnée N'EST PAS supprimée après consommation. La rétention est gérée par segment : les vieux segments sont supprimés ou compactés.

Rétention par temps

```
log.retention.hours=168
```

Messages conservés N heures. Standard : 7 jours. Peut aller à des années. Passé le délai → vieux segments supprimés.

Log Compaction

```
cleanup.policy=compact
```

Garde uniquement le DERNIER message par clé. Idéal : KTable, CDC, configurations. État courant permanent.

Rétention par taille

```
log.retention.bytes=1073741824
```

Partition limitée à N bytes. Vieux segments supprimés en premier. -1 = illimité. Utile pour maîtriser le disque.

Compact + Delete

```
cleanup.policy=compact,delete
```

Combine : compacte (1 msg/clé) puis supprime selon rétention temporelle. Kafka Streams changelog topics.

Modifier : kafka-topics.sh --alter --topic orders --config retention.ms=2592000000 (30 jours en ms)

Performance Kafka — Benchmarks et leviers d'optimisation

Scénario	Débit	Latence P99	Config
Producer acks=1 + snappy	**>500 MB/s**	~2ms	batch.size=64KB · linger.ms=5
Producer acks=all + snappy	**>200 MB/s**	~8ms	acks=all · min.ISR=2 · RF=3
Consumer (zero-copy)	**>1 GB/s**	~2ms	fetch.min.bytes=64KB
Producer acks=all idempotent	**>150 MB/s**	~10ms	enable.idempotence=true

Levier	Configuration	Gain attendu
Compression snappy/lz4	**compression.type=snappy**	-50 à -70% réseau · overhead CPU minimal
Batch size + linger	**batch.size=65536 + linger.ms=5**	4× moins d'appels réseau · débit 3× plus élevé
Threads broker	**num.io.threads=16 + num.network.threads=8**	Parallélisme I/O optimal pour multi-core

Métriques de réplication à surveiller en production

UnderReplicatedPartitions

Seuil : = 0 (CRITIQUE si > 0)

Partitions avec réplicas en retard. Toute valeur > 0 = problème de réplication actif. Alarme critique immédiate.

OfflinePartitionsCount

Seuil : = 0 (ALARME MAJEURE)

Partitions sans leader actif = service dégradé. Tous les ISR sont tombés. Action immédiate.

IsrShrinkRate

Seuil : → 0 en état stable

Taux de sortie ISR. Activity > 0 = réseau ou IO instable. IsrExpandRate = retour en ISR.

BytesInPerSec

Seuil : < 80% capacité

Débit entrant par broker. Si > 80% → risque saturation réseau. Capacity planning.

LeaderCount

Seuil : Équilibré entre brokers

Leaders par broker. Idéalement équilibré. Déséquilibre = charge inégale. Corriger avec preferred leader election.

ReplicationBytesInPerSec

Seuil : ≈ BytesIn × (RF-1)

Trafic réplication normal = BytesIn × (RF-1). Anomalie = réplication partielle.

Quiz — Section 03 : Réplication & Haute Disponibilité

❓ RF=3, min.ISR=2 : combien de brokers peuvent tomber sans perte ?

- ○ 0 — tous nécessaires
- ✔ **1 broker peut tomber sans perte de données**
- ○ 2 brokers simultanément
- ○ 3 brokers

❓ UnderReplicatedPartitions > 0 signifie :

- ○ Normal pendant quelques secondes
- ✔ **Des partitions n'ont pas assez de réplicas ISR — alarme critique**
- ○ Kafka crée des réplicas auto
- ○ Le cluster redémarre

❓ unclean.leader.election.enable=false signifie :

- ○ Impossible de réélire les leaders
- ✔ **Seuls les brokers ISR peuvent devenir leader — priorité durabilité**
- ○ Réplication désactivée
- ○ Kafka s'arrête si broker tombe

❓ Quelle stratégie garde 1 message par clé ?

- ○ cleanup.policy=delete
- ✔ **cleanup.policy=compact**
- ○ log.retention.bytes=-1
- ○ log.retention.hours=0

04

KRaft — Architecture sans ZooKeeper

Kafka 3.3+ GA · Controller Quorum Raft · Configuration · Migration

ZooKeeper vs KRaft — Comparatif complet

Critère	Mode ZooKeeper (déprécié)	Mode KRaft — Kafka 3.3+ GA
Composants	Kafka + ZooKeeper (2 systèmes)	**Kafka seul — 1 système intégré**
Démarrage	Plus lent — ZK doit démarrer en premier	**2× plus rapide — tout démarre ensemble**
Partitions max	~200 000 partitions	**10 000 000+ partitions**
Latence metadata	50–200ms (via réseau ZK)	**< 10ms (intégré, même JVM)**
Rolling restart	Complexe — ZK puis Kafka	**Simplifié — un seul système**
Monitoring	Dual : surveiller ZK + Kafka	**Unique : Kafka seulement**
TLS / ACLs	Config double ZK + Kafka	**Config unique server.properties**

KRaft — Architecture interne et protocole Raft

⚙️ Protocole KRaft

KRaft (Kafka + Raft) utilise le consensus Raft pour élire un Active Controller et répliquer le metadata log. Les métadonnées sont dans __cluster_metadata — un topic Kafka interne immuable.

Active Controller

Broker élu leader du quorum via Raft. Gère toutes les opérations metadata : création topics, élection leaders partitions, gestion brokers. 1 seul active controller.

Metadata Log

Topic interne __cluster_metadata : log immuable de toutes les opérations config. Chaque broker a une copie locale → accès metadata < 1ms.

Controller Quorum

3 brokers votent via Raft. Quorum = majorité (2/3). Si controller tombe → nouveau élu en quelques secondes. Tolérance : 1 panne sur 3.

Snapshot

Périodiquement le controller crée un snapshot du metadata log pour éviter sa croissance infinie. Brokers bootstrappent depuis snapshot + delta.

Configuration KRaft — server.properties complet

server.properties — Broker 1 KRaft (mode combiné)

```
######################### KRaft Mode #######################
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@broker1:9093,2@broker2:9093,3@broker3:9093

# Listeners
listeners=PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
advertised.listeners=PLAINTEXT://broker1:9092
inter.broker.listener.name=PLAINTEXT
controller.listener.names=CONTROLLER

################### Production Settings #################
# Réplication (durabilité)
default.replication.factor=3
min.insync.replicas=2
offsets.topic.replication.factor=3
transaction.state.log.replication.factor=3

# Sécurité
unclean.leader.election.enable=false  # ACTIVER EN PROD

# Stockage
log.dirs=/var/kafka/data
log.retention.hours=168
log.segment.bytes=1073741824  # 1GB par segment

# Performance
num.network.threads=8
num.io.threads=16
```

Initialisation du cluster

Étape 1 — Générer Cluster ID

```
$ kafka-storage.sh random-uuid
→ MkU3OEVBNTcwNTJENDM2Qk
```

Étape 2 — Formater disques

```
$ kafka-storage.sh format \
  -t MkU3OEVBNTcwNTJENDM2Qk \
  -c /etc/kafka/server.properties
```

Étape 3 — Démarrer brokers

```
$ kafka-server-start.sh \
  /etc/kafka/server.properties
```

Modes de déploiement KRaft — Combiné vs Séparé

Mode Combiné (Combined)

process.roles=broker,controller · Chaque broker est à la fois broker ET controller. Idéal dev/staging, petits clusters.

✔ Avantages

- Moins de machines requises
- Déploiement simplifié
- Parfait pour Docker/k8s local

✗ Contraintes

- Contention broker/controller
- Moins adapté très grands clusters
- GC pauses peuvent impacter quorum

```
process.roles=broker,controller
```

Mode Séparé (Isolated)

Controllers dédiés uniquement à la gestion des métadonnées. Brokers ont uniquement le rôle broker. Recommandé production à grande échelle.

✔ Avantages

- Ressources dédiées par rôle
- Performances optimales grande échelle
- Isolation GC resources

✗ Contraintes

- Plus de machines (3 ctrl + N brokers)
- Déploiement plus complexe
- Coût infrastructure légèrement plus élevé

```
Controllers: process.roles=controller
Brokers:     process.roles=broker
```

Recommandation : Combiné pour dev/test (<10 brokers) · Séparé pour production (3 controllers + N brokers données)

Migration ZooKeeper → KRaft — Guide pratique

⚠️ **Avant de migrer**

Migration hors-downtime (rolling). Nécessite Kafka 3.4+. Tester en staging AVANT production. Lire KIP-833 officiel.

1 — Prérequis

Kafka 3.4+ tous brokers · Sauvegarder ZooKeeper data · Tester en staging · Lire KIP-833

2 — Déployer Controllers

3 nœuds process.roles=controller · Configurer controller.quorum.voters · Lancer les controllers KRaft

3 — Migration metadata

kafka-storage.sh --zookeeper-connect → exporter métadonnées ZK vers metadata log KRaft

4 — Rolling restart

Redémarrer chaque broker avec zookeeper.metadata.migration.enable=true · Un par un · Valider entre chaque

5 — Finalisation

Supprimer ZooKeeper des configs · kafka-features.sh finalize · Valider kafka-metadata-quorum.sh

KRaft — Monitoring et commandes opérationnelles

Statut du quorum KRaft

```
kafka-metadata-quorum.sh \
  --bootstrap-server localhost:9092 \
  describe --status
# Résultat attendu :
# ClusterId:       MkU3OEVBNTcwNTJENDM2Qk
# LeaderId:        1
# LeaderEpoch:     5
# HighWatermark:   1842
# MaxFollowerLag:  0     ← idéal = 0
# CurrentVoters:   [1,2,3] ← 3 controllers
```

Santé du cluster

```
# Features KRaft activées
kafka-features.sh \
  --bootstrap-server localhost:9092 \
  describe

# API versions des brokers
kafka-broker-api-versions.sh \
  --bootstrap-server localhost:9092

# Controller actuel
kafka-topics.sh --describe \
  --topic __cluster_metadata
```

Inspecter le metadata log

```
kafka-metadata-shell.sh \
  --snapshot /var/kafka/data/__cluster_metadata-0/
# Commandes disponibles :
ls /brokers/        # liste les brokers
cat /topics/orders/ # config du topic
ls /acls/           # ACLs configurées
cat /features/      # feature flags
```

Snapshots metadata

```
ls /var/kafka/data/__cluster_metadata-0/
# → 00000000000000000000.log
# → 00000000000000001000.checkpoint
# → 00000000000001000000.snapshot

# Config fréquence snapshot :
metadata.log.max.record.bytes
  .between.snapshots=20000000
```

KRaft : 1 seul système, 1 seul set de configs, 1 seul monitoring — simplification opérationnelle majeure.

Quiz — Section 04 : KRaft & Architecture

❓ Principale amélioration de KRaft vs ZooKeeper ?

- ○ Plus rapide pour écrire des messages
- ✔ **Intègre la gestion métadonnées dans Kafka — plus de composant séparé**
- ○ Supporte plus de formats
- ○ Supprime le Schema Registry

❓ process.roles en mode combiné doit valoir :

- ○ broker
- ○ controller
- ✔ **broker,controller**
- ○ metadata,broker

❓ Topic interne qui stocke les métadonnées KRaft ?

- ○ __consumer_offsets
- ✔ **__cluster_metadata**
- ○ __transaction_state
- ○ __kafka_metadata

❓ Nb controllers KRaft pour tolérer 1 panne ?

- ○ 1 controller
- ○ 2 controllers
- ✔ **3 controllers (quorum 2/3)**
- ○ 5 minimum

05

TP — Déploiement, CLI & Exercices pratiques

Cluster Docker Compose · kafka-topics · CLI complète · 5 exercices progressifs

TP 1 — Docker Compose : Cluster Kafka 3 brokers KRaft

docker-compose.yml — Kafka 3 brokers KRaft + Kafka UI

```yaml
version: '3.8'
services:
  broker1:
    image: confluentinc/cp-kafka:7.6.0
    hostname: broker1
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: >-
        1@broker1:9093,2@broker2:9093,3@broker3:9093
      KAFKA_LISTENERS: >-
        PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://broker1:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
      KAFKA_DEFAULT_REPLICATION_FACTOR: 3
      KAFKA_MIN_INSYNC_REPLICAS: 2
      CLUSTER_ID: "MkU3OEVBNTcwNTJENDM2Qk"
    ports: ["19092:9092"]
    volumes: ["broker1-data:/var/lib/kafka/data"]
  broker2: # idem · NODE_ID: 2 · port 19093
  broker3: # idem · NODE_ID: 3 · port 19094
  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    ports: ["8080:8080"]
    depends_on: [broker1, broker2, broker3]
    environment:
      KAFKA_CLUSTERS_0_NAME: formation-cluster
```

Commandes de démarrage

Cloner le repo

```
$ git clone <url-formation>
```

Aller dans le dossier

```
$ cd kafka-formation/jour1
```

Démarrer tout

```
$ docker compose up -d
```

Vérifier conteneurs

```
$ docker compose ps
```

Logs broker1

```
$ docker logs broker1 -f
```

Shell broker1

```
$ docker exec -it broker1 bash
```

✔ **Kafka UI**
http://localhost:8080

docker-compose.yml complet disponible dans le repo Git — docker compose up -d démarre tout en 30 secondes.

TP 2 — CLI kafka-topics : Créer et gérer les topics

kafka-topics.sh — Référence complète

```
# ─── CRÉER UN TOPIC ──────────────────────────
kafka-topics.sh \
  --bootstrap-server localhost:19092 \
  --create --topic commerce.orders \
  --partitions 6 \
  --replication-factor 3 \
  --config retention.ms=604800000 \
  --config min.insync.replicas=2

# ─── LISTER ──────────────────────────────────
kafka-topics.sh \
  --bootstrap-server localhost:19092 --list

# ─── DÉCRIRE (voir partitions + ISR) ─────────
kafka-topics.sh \
  --bootstrap-server localhost:19092 \
  --describe --topic commerce.orders
# Partition: 0  Leader: 1  Replicas: 1,2,3
# Isr: 1,2,3

# ─── MODIFIER ────────────────────────────────
# Augmenter partitions (irréversible !)
kafka-topics.sh --alter \
  --topic commerce.orders --partitions 12

# Changer rétention
kafka-configs.sh --alter \
  --entity-type topics \
  --entity-name commerce.orders \
```

📋 Exercice guidé

1 — Créer les topics du projet

- commerce.orders (6P RF=3 min.ISR=2)
- commerce.events (3P RF=3)
- commerce.dlq (1P RF=2)
- analytics.metrics (6P RF=3)

2 — Explorer la distribution

- Décrire commerce.orders
- Noter broker leader de P0
- Vérifier Isr: 1,2,3
- Kafka UI → onglet Topics

3 — Observer le cluster

- Identifier controller actif
- Lister tous les brokers
- UnderReplicatedPart = 0 ?
- Kafka UI → onglet Brokers

✔ Résultat : 4 topics · Isr: 1,2,3
Leaders répartis · Kafka UI affiche tout

TP 3 — CLI Producer & Consumer — Pub/Sub en action

kafka-console-producer.sh

```
kafka-console-producer.sh \
  --bootstrap-server localhost:19092 \
  --topic commerce.orders \
  --property parse.key=true \
  --property key.separator=|
> order-1|{"id":1,"status":"CREATED","amount":99.99}
> order-2|{"id":2,"status":"CREATED","amount":49.00}
> order-1|{"id":1,"status":"PAID","amount":99.99}
> order-3|{"id":3,"status":"CREATED","amount":149.00}
^C
```

kafka-consumer-groups.sh — Lag

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:19092 \
  --describe --group groupe-A
# TOPIC           PARTITION  OFFSET  END  LAG
# commerce.orders 0          2       2    0
# commerce.orders 1          1       1    0
# ... (6 partitions)

# Reset — replay depuis offset 0
kafka-consumer-groups.sh \
  --group groupe-A \
  --topic commerce.orders \
  --reset-offsets --to-earliest --execute
```

kafka-console-consumer.sh

```
# Consumer Groupe A  (Terminal 2)
kafka-console-consumer.sh \
  --bootstrap-server localhost:19092 \
  --topic commerce.orders \
  --group groupe-A --from-beginning \
  --property print.key=true \
  --property print.offset=true

# Consumer Groupe B indépendant  (Terminal 3)
kafka-console-consumer.sh \
  --bootstrap-server localhost:19092 \
  --topic commerce.orders \
  --group groupe-B --from-beginning
```

Exercice : 3 terminaux ouverts

```
# T1 — Producer continu (broker1 shell)
kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic commerce.orders \
  --property parse.key=true \
  --property key.separator=|

# T2 — Consumer Groupe A
kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --group groupe-A --from-beginning

# T3 — Consumer Groupe B (indépendant)
```

Observer : les 2 groupes reçoivent indépendamment les mêmes messages → Pub/Sub natif dans Kafka !

EX 1 — Déploiement et exploration du cluster (20 min)

🎯 **Objectif : Déployer le cluster 3 brokers et explorer l'interface Kafka UI**

1 Cloner et démarrer

- git clone <url-repo>
- cd kafka-formation/jour1
- docker compose up -d
- docker compose ps → 3 brokers + kafka-ui Up

2 Explorer Kafka UI

- Ouvrir http://localhost:8080
- Onglet Brokers → 3 brokers Online
- Onglet Topics → voir __consumer_offsets
- Cliquer sur chaque broker → voir ses partitions

3 Vérifier quorum KRaft

- docker exec -it broker1 bash
- kafka-metadata-quorum.sh ... describe --status
- Vérifier MaxFollowerLag=0
- CurrentVoters=[1,2,3]

4 Vérifier les logs

- docker logs broker1 | grep started
- Chercher : Kafka Server started
- Aucune ERROR dans les logs
- Vérifier broker.id=1 dans les logs

EX 2 — Création et gestion des topics (25 min)

🎯 **Objectif : Créer les topics du projet fil rouge et comprendre la distribution des partitions**

Commandes à exécuter

```
docker exec -it broker1 bash

# Créer les 4 topics du projet
kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic commerce.orders \
  --partitions 6 --replication-factor 3 \
  --config min.insync.replicas=2

kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic commerce.events \
  --partitions 3 --replication-factor 3

kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic commerce.dlq \
  --partitions 1 --replication-factor 2

kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic analytics.metrics \
  --partitions 6 --replication-factor 3

# Lister et décrire
kafka-topics.sh --list
kafka-topics.sh --describe --topic commerce.orders
# → Vérifier : Isr: 1,2,3 pour chaque partition
# → Vérifier : leaders répartis sur 3 brokers
```

📋 Points à valider

- ☐ 4 topics créés (kafka-topics --list)
- ☐ commerce.orders : 6 partitions
- ☐ Isr=[1,2,3] pour chaque partition
- ☐ Leaders équilibrés (brokers 1, 2, 3)
- ☐ Kafka UI affiche les 4 topics
- ☐ commerce.dlq : RF=2 seulement

EX 3 — Produce & Consume en temps réel (30 min)

🎯 **Objectif : Pub/Sub en action · 2 Consumer Groups indépendants · Observer les offsets**

Terminal 1 — Producer

```
docker exec -it broker1 kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic commerce.orders \
  --property parse.key=true --property key.separator=|
> order-1|{"id":1,"status":"CREATED","amount":99.99}
> order-2|{"id":2,"status":"CREATED","amount":49.00}
> order-1|{"id":1,"status":"PAID","amount":99.99}
```

Terminal 2 — Consumer Groupe A

```
docker exec -it broker1 kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic commerce.orders \
  --group groupe-A --from-beginning \
  --property print.key=true --property print.offset=true
```

Terminal 3 — Consumer Groupe B

```
docker exec -it broker1 kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic commerce.orders \
  --group groupe-B --from-beginning
```

🔍 Observations

- ✔ Groupes A et B reçoivent les mêmes msgs
- ✔ Chaque groupe a ses propres offsets
- ✔ Ajouter msg T1 → T2 ET T3 mis à jour
- ✔ Ctrl+C sur T2 → T3 continue
- ✔ kafka-consumer-groups --list → 2 groupes
- ✔ LAG = 0 si consumers à jour
- ✔ Kafka UI → Consumer Groups → 2 groupes

EX 4 — Résilience : Simulation de panne broker (25 min)

🎯 **Objectif : Observer la résilience Kafka · Réélection automatique · RTO < 30s**

1 — Identifier le leader

```
$ kafka-topics.sh --bootstrap-server localhost:19092 --describe --topic
commerce.orders
```

→ → Notez le Leader de Partition 0 (ex: Leader: 1)

2 — Arrêter le broker leader

```
$ docker stop broker1   # si broker1 est leader de P0
```

→ → Simule une panne matérielle soudaine

3 — Observer la réélection

```
$ kafka-topics.sh --bootstrap-server localhost:19093 --describe --topic
commerce.orders
```

→ → Broker 2 ou 3 est nouveau leader · ISR=[2,3]

4 — Vérifier continuité

```
$ kafka-console-producer.sh --bootstrap-server localhost:19093 --topic
commerce.orders
```

→ → Production continue sur le nouveau leader !

5 — Redémarrer broker1

```
$ docker start broker1 && sleep 30
```

→ → Rejoue les messages manqués · ISR redevient [1,2,3]

Mesurer le RTO : temps entre docker stop et la réélection effective (typiquement < 30 secondes).

EX 5 — Consumer Groups : Parallélisme et rebalancing (20 min)

🎯 **Objectif : Parallélisme des Consumer Groups et rebalancing en action**

Séquence complète à réaliser

```
# Étape 1 — 3 consumers dans le même groupe
# Terminal 1 :
docker exec -it broker1 kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic commerce.orders --group perf-group

# Terminal 2 (même groupe) :
docker exec -it broker1 kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic commerce.orders --group perf-group

# Terminal 3 (même groupe) :
docker exec -it broker1 kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic commerce.orders --group perf-group

# Étape 2 — Observer l'assignation
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe --group perf-group
# → Voir partitions assignées à chaque consumer

# Étape 3 — Stopper Terminal 2 (Ctrl+C)
# → Observer rebalancing dans docker logs broker1
```

📊 Résultats attendus

3 consumers, 6 partitions
→ 2 partitions par consumer

Consumer tombe
→ Partitions redistribuées auto

6 consumers, 6 partitions
→ 1 partition / consumer = optimal

7 consumers
→ Le 7ème est IDLE

kafka-consumer-groups
→ LAG = 0 si à jour

Règle d'or : nb consumers actifs = min(nb consumers lancés, nb partitions du topic).

Récapitulatif Jour 1 — 6 concepts + Aperçu Jour 2

📐 Log Distribué immuable

Append-only, persisté disque, replay depuis n'importe quel offset. Différent d'une queue classique.

🛡️ Réplication RF=3 + ISR

Leader gère R/W. acks=all + min.ISR=2 = zéro perte. Tolère crash simultané de 1 broker.

🗂️ Partition = scalabilité

Unité de parallélisme. hash(key)%N pour l'ordre. Impossible à réduire. Dimensionner dès le départ.

⚙️ KRaft sans ZooKeeper

Metadata interne via Raft. 1 système. 10M partitions. Démarrage 2× plus rapide.

🔄 Consumer Group

Nb consumers ≤ nb partitions. 1 partition = 1 consumer par groupe max. Rebalancing automatique.

🖥️ CLI complète maîtrisée

kafka-topics · kafka-console-producer/consumer · kafka-consumer-groups · kafka-configs